Université Côte d'Azur
Licence Électronique
Enseignement : SLEEL602 - Microprocesseurs

TD2

Modes d'adressage et branchements conditionnels

1 Branchements conditionnels

1.1 Introduction

Considérons l'extrait de code suivant :

```
  LDI  R16, 10
  CPI  R16, 9
  BRNE test
  SUBI R16, 2
  LSL  R16
test:
  LDI  R17, 6
  ADD  R16, R17
  LSL  R16
  SUBI R16, 1
```

R17	R16	Z	N	C
/	10	/	/	/
/	10	0	0	0
6	10	/	/	/
6	16	0	0	0
6	32	0	0	0
6	31	0	0	0

Donnez la valeur des registres ainsi que la valeur de l'indicateur Z du registre d'état SREG pour toutes les instruction qui doivent être exécutées.

1.2 Structures de contrôle

1. Soit l'algorithme suivant (on va considérer que la variable A est stockée en mémoire à l'adresse 0) :

```
r18 <-- A
si (r18 <= 10) alors
   r20 <-- 12
sinon
   r25 <-- 30
finsi
```

Représentez cet algorithme en instructions assembleur ATmega2560.

```
    ld   R18, 0
    cpi  R18, 10
    BRLT test
    CPI  R18, 10
    BREQ test
         R25, 30
         Jump finsi
test:
    ld R20, 12
finsi
```

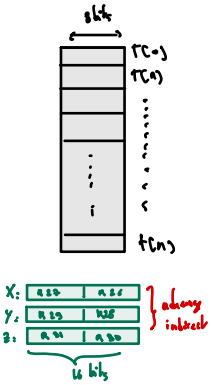
loop: cpi r16, 0 ; Comparer r16 avec 0 brlt end_loop ; Si r16 < 0, sortir de la boucle ldi r22, 5 ; Charger 5 dans r22 add r22, r16 ; r22 = r16 + 5 dec r16 ; r16 = r16 - 1 rjmp loop ; Répéter la boucle end_loop: nop ; Instruction NOP (fin du programme)

2. On va étudier maintenant les structures qui permettent de réaliser des boucles. Considerons l'algorithme suivant :

```
r16 <-- 12
tant que (r16 >= 0) faire
   r22 <-- r16 + 5
   r16 <-- r16 - 1
fin tant que
```

Représentez cet algorithme en instructions assembleur ATmega2560.

3. Pour ceux qui sont assez experts en assembleur ATmega2560, vous pourrez considérer cet exercice supplémentaire. Considérons l'algorithme suivant qui détermine la plus grande valeur dans un tableau. Ces valeurs sont toutes codées sur 8 bits et signées.



```
max <-- t[0]
pour tout n de 1 jusqu'a NMAX
  si max < t[n] alors
    max <-- t[n]
  finsi
fin pour
```


Écrivez le code assembleur pour la machine à registres généraux correspondante.

2 Mode d'adressage

2.1 Exercice 1

Soit a une donnée codée sur 16 bits rangée à l'adresse mémoire \$0100. Écrivez en pseudo-code et en assembleur AVR ATmega2560 la liste des instructions assembleurs nécessaires pour ranger en mémoire la valeur $b = a + 15$ à la suite de a en utilisant l'adressage :

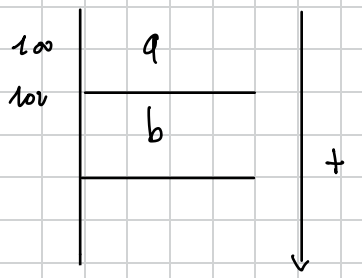
— direct
— indirect
— indirect avec déplacement constant
— indirect avec post-incrémentation

2.2 Exercice 2

Dans l'assembleur que vous utiliserez en TP, on va pouvoir utiliser des macros qui permettront de nous faciliter énormément les choses :

1. Pour les calculs des valeurs sur 8 bits, vous allez pouvoir utiliser les expressions $lo8(V)$ et $hi8(V)$ afin d'extraire la valeur 8 bit correspondant au poids faible de la variable V et au poids fort de la variable V
2. Les adresses particulières des registres d'entrée/sortie. Ces registres seront utilisables (pour toutes les adresses coprises entre $0x20$ et $0x5F$) grâce à l'expression $_SFR_IO_ADDR()$ avec le nom du registre comme argument. Par exemple, cette

a. données 16 bits $0100, range les valeurs b = a + 15 à les adresses de a même data.



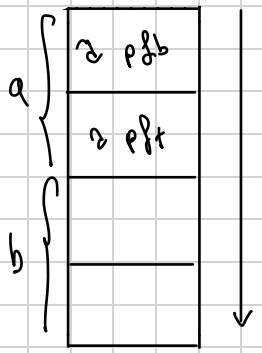
adressage direct:

R16 ← mem [100]
R17 ← mem [101]
R18 ← 15_{16}
R16 ← R16 + R18
R19 ← 0
R17 ← R17 + C + R19
R17, R16 ← R17, R16 + R18
mem [102] ← R16
mem [103] ← R19.

ADIW → add → 2 registres

a = 0B F2
R17 R16

b. a, 16 bits, 2 entrées à l'adresse $x100 : Indirect X, Y, Z
b = a + 15 à l'adresse suivante.



adressage indirect

Y → correspond au 2 registres
R28, R29
pfb pft

0x100 → R28

R28 ← 00 ← pfb
R29 ← 1 ← pft

R16 ← mem [Y] pfb
R28 ← R28 + 1
R17 ← mem [Y]

R17, R16 ← R17, R16 + 15

R18 ← 15
R28 ← R28 + R18
R19 ← 0
R19 ← R19 + C + R19

R28 ← R28 + 1
mem [Y] ← R16
R28 ← R28 + 1
mem [Y] ← R17

ldi R18, 15
ldi R19, 00 XL
ldi R19, 1 XH

ldd R16, Y
INC R28
ldd R17, Y

Adiw R16, R18

INC R28
STd Y, R16
INC R28
std Y, R17

adressage indirect déplacement Cst.

a/ R28 ← 00
R29 ← 1

R16 ← mem [Y]
R17 ← mem [Y+1]

R17, R16 ← R17, R16 + 15

mem [Y+2] ← R16
mem [Y+3] ← R17

```
ldi R18, 15
ldi R19, 00   XL
ldi R19, 1    XH

ld  R16, Y
ld  R17, Y + 1

Adiw R16, R18

St Y+2, R16
st Y+3, R17
```

mode post-inc

R28 ← 0
R29 ← 1

Rd ← mem [Y+]
Rd ← mem [Y]
Y ← Y+n

R16 ← mem [Y+]
R17 ← mem [Y+]

R17, R16 ← R17, R16 + 15
mem [Y+3] ← R16
mem [Y+] ← R17

```
ldi R18, 15
ldi R19, 00   XL
ldi R19, 1    XH

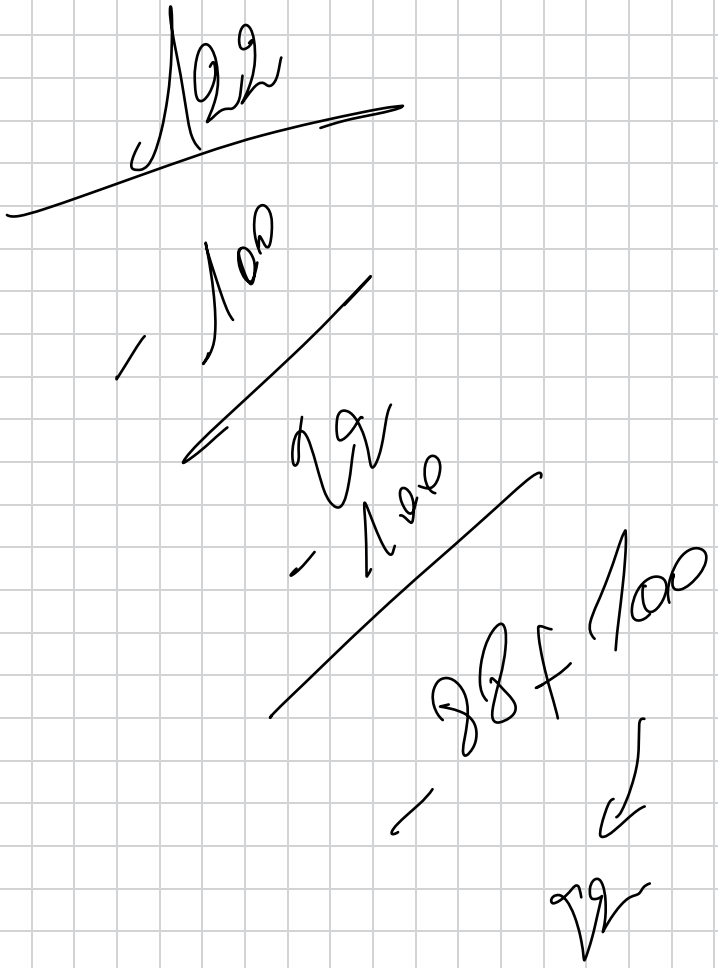
ld  R16, Y
ld  R17, Y +

Adiw R16, R18
St Y+, R16
st Y+, R17
```

chaine : .asciz "bonjour"

main : ldi R26, ls 8(chaine)
ldi R27, hi 8(chaine)
(X ← chaine)

Push R21
Push R22
Push R23



expression $_SFR_IO_ADDR(DDRF)$ représente l'adresse du registre DDRF soit la valeur $0x30$.

1. Écrivez le programme qui permet de faire l'initialisation et les instructions de base pour l'allumage d'une LED qui serait branchée sur la ligne 4 du port A.
2. Écrivez le programme qui permet de contrôler l'allumage et l'extinction de la LED grâce à un interrupteur branché sur la ligne 6 dur port C.